

35.5. La fonction `cachier32()` — ôter un grand personnage

`#include <cachier32>` : `cachier32(numero)` ôte les seize lutins d'un grand personnage de 32×32 .

1. Ce qu'il faut comprendre

Ce qu'elle fait : `cachier32(numero)` ôte les seize lutins `numero` à `numero + 15` : tout le grand personnage.

La ligne à écrire : `#include <cachier32>`.

Son argument : le numéro du premier lutin, comme pour `sprite32()`.

Essaie : fais-le réapparaître à l'image 240.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **872 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      47 o ← la console, à l'allumage – la mise en route de la console
main()         450 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
RangerLutins   11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
Tuiles         256 o ← CopierTuiles – 0 lettre sur 43, et 16 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert – la petite routine qui copie les lutins
```

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

```
sprite()   x 16   384 o sur place, dans main()
cachier32() x 1    49 o sur place, dans main()
image()    x 1     6 o sur place, dans main() + AttendreImage (28 o)
images()   x 1     2 o sur place, dans main()
sprite16() x 4     0 o sur place, dans main()
sprite32() x 1     0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <cachier32> : ôter un grand personnage de 32 x 32 ----
// GEANT est dessiné dans perso_GEANT.cpp ; ici, on ne fait que l'appeler.
```

```
#include <sprite32>           // sprite32() : le poser
#include <cachier32>          // cachier32() : ôter ses seize lutins
#include "personnages.cpp"    // la liste : GEANT
```

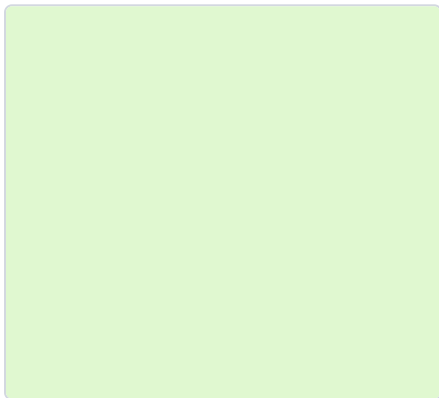
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    sprite32(0, 64, 56, GEANT);    // les lutins 0 à 15

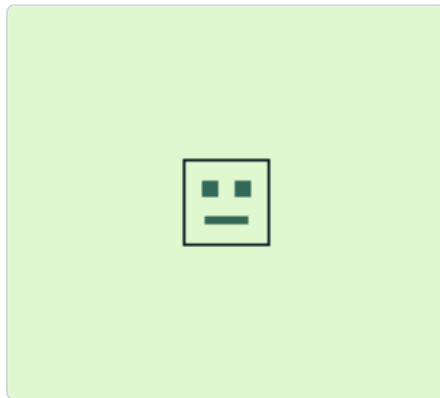
    while (true) {
        image();
        if (images() == 120) {
            cacher32(0);            // les lutins 0 à 15 s'en vont
        }
    }
}
```

3. Ce qui se passe, étape par étape

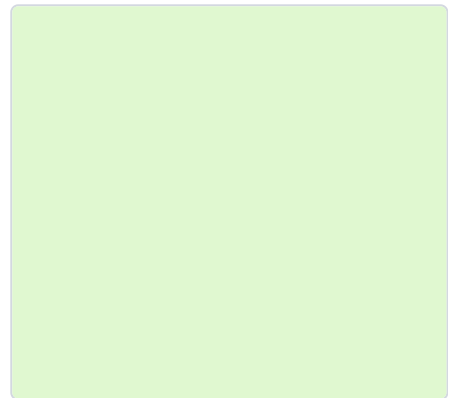
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce qu'on doit voir

La grande tête de robot (GEANT) au milieu, qui disparaît au bout de deux secondes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, GEANT est là (88, 80)
- ✓ puis ses seize lutins sont hors de l'écran

Ensuite : **35.6. BONHOMME, de son fichier à l'écran : `sprite16()`** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++